TriVoKo

Multi-seller E-commerce Marketplace

Final Guide and Viva Q&A

The finished project in simple words - architecture, the key flows, testing, launch - and 40 interview questions with short answers



Madhavan Suresh

Java Full Stack main portfolio project · React + Spring Boot + MySQL + Stripe

Version 1.0 · 4 November 2026 · Phase 12 launch

Contents

1. TriVoKo in two minutes 3
1.1 The 20-second answer 3
1.2 The two-minute answer 3
1.3 The numbers to remember 4
1.4 The part you are most proud of 4
2. How TriVoKo is built 5
2.1 The big picture 5
2.2 Frontend (React) 5
2.3 Backend (Spring Boot) - a modular monolith 6
2.4 Database 6
2.5 Security 7
3. The important flows, step by step 8
3.1 Buying from two sellers with one payment 8
3.2 A seller ships a package 8
3.3 The flash sale 9
3.4 A return and its partial refund 9
3.5 Price-drop alerts 10
3.6 Smart search 10
3.7 One-click demo logins (launch) 10
4. How we know it works 11
4.1 The test pyramid 11
4.2 CI - GitHub Actions on every push 11
4.3 Performance and accessibility 11
5. How it is launched 12
5.1 The three free services 12
5.2 Making Spring Boot fit in 512 MB 12
5.3 Production settings 12
5.4 What a launch taught us 12
6. What I would do next 13
7. Viva and interview questions 14
7.1 The project 14
7.2 Java and Spring Boot 14
7.3 Database and concurrency 15
7.4 Security 15
7.5 Payments 16
7.6 React 16

7.7 Testing and DevOps 17
7.8 Smart search and alerts 17
7.9 Reflection 18

1. TriVoKo in two minutes

This guide is written for **you, MADHAVAN**, to read before an interview or a viva. Chapters 1-6 explain the finished project in simple words. Chapter 7 is a list of questions an interviewer may ask, each with a short answer you can say in your own words.

> **Real-life example:** When a friend asks "what does your shop sell?", you do not read the stock register to them. You say one sentence, then two or three things that make your shop special. This chapter is that sentence and those things.

1.1 The 20-second answer

"TriVoKo is a multi-seller marketplace - a small Flipkart - that I built from an empty folder to a live website with React and Spring Boot. A customer fills one cart from many shops and pays once; the order is split into one package per seller, and each seller packs and ships their own package. I added four harder features: a flash sale that never oversells, returns with exact partial refunds, price-drop alerts, and a typo-tolerant search."

1.2 The two-minute answer

Say it in this order - **problem, product, four standouts, quality, live**:

- **Problem.** A marketplace is harder than a single shop: one payment must be split between many sellers, and money, stock and each seller's earnings must always agree.
- **Product.** Three roles. Customers search, fill one cart, pay once and track one package per seller. Sellers - approved by the admin - list products with sizes and colours, ship, and see their earnings (90%; TriVoKo keeps 10%). The admin approves shops and products, runs flash sales and coupons, and reads reports.
- **Four standouts.**
  - **Flash sale:** 100 units, 5,000 people - exactly 100 winners. One atomic SQL `UPDATE ... WHERE sold < quantity`; proven with a k6 load test.
  - **Returns:** a state machine and a Stripe **partial** refund of exactly the returned item minus its share of the coupon.
  - **Price-drop alerts:** one alert per real drop, never spam - an event marks rows, a job sends the alert.
  - **Smart search:** "iphnoe" finds the iPhone case - Hibernate Search with Lucene, with counts for brands and prices.
- **Quality.** About 400 backend tests on a real MySQL (Testcontainers), 100+ React tests, a Playwright test that clicks through the whole story in a real browser, accessibility and security scans - all in GitHub Actions on every push.
- **Live.** It runs in Docker on Render, with MySQL on Aiven and the React app on Vercel - all on free plans - and a recruiter can log in with one click as a demo customer or seller.

1.3 The numbers to remember

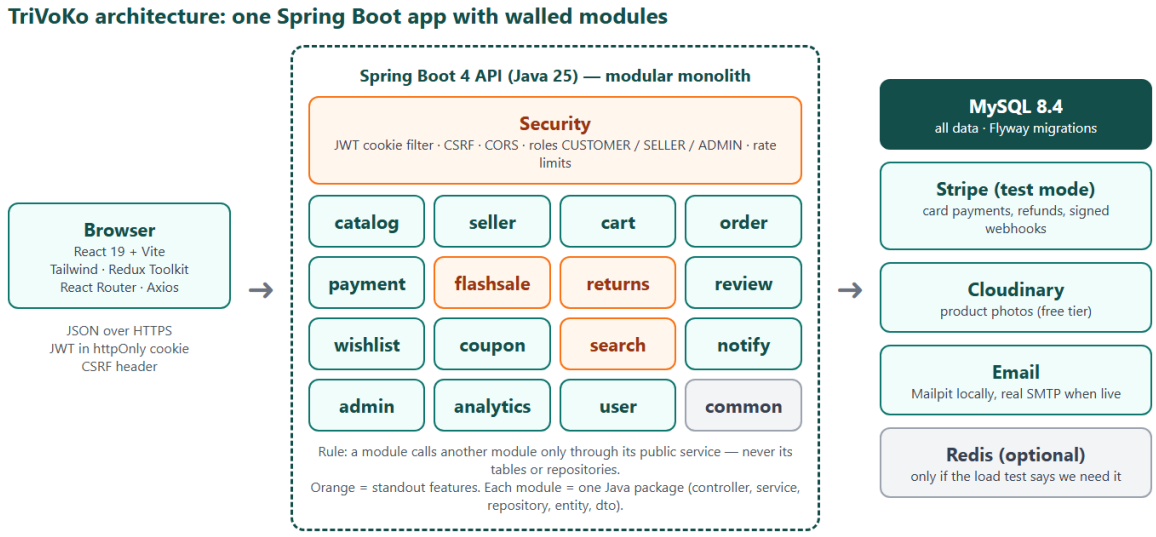
Number	What it means
13 phases	Phase 0 (setup) to Phase 12 (launch), one Git branch and pull request per phase
17 modules	`catalog`, `order`, `payment`, `flashsale`, `returns`, `search` ... in one Spring Boot app
12 migrations	Flyway files V1-V12 build every table; Hibernate only checks them
100 of 5,000	flash-sale load test: 100 units, 5,000 users, exactly 100 winners, 0 errors, 0 oversell
91.7%	line coverage of the service classes; the build fails below 80%
92	Lighthouse performance score of the home page on a phone (accessibility 100)
Rs 0	hosting cost: Render + Aiven + Vercel free plans

1.4 The part you are most proud of

Pick one and be ready to explain it deeply (chapters 3 and 7 help). A good choice is the **flash sale**, because it has a clear risk (overselling), a clear one-line fix (the atomic update), and a clear proof (the load test with exactly 100 winners). Another good choice is the **order split**, because you can draw it on a whiteboard in one minute.

2. How TriVoKo is built

2.1 The big picture



Architecture

```
Browser (phone / laptop)
   |  https://trivoko.vercel.app  - one address
   v
Vercel ── React app (built by Vite)
   |  /api/*  is passed on (rewrite)
   v
Render ── Spring Boot 4 in Docker (Java 25, 512 MB memory)
   |      17 modules, Lucene search index rebuilt on start
   v
Aiven ── MySQL 8.4 (tables built by Flyway)
```

Why one address? The browser only talks to Vercel. Vercel quietly passes every `/api/...` call to Render. So for the browser the shop and the API are the same website, and the login cookie works as a normal first-party cookie - no cross-site cookie problems, no CORS in normal use.

2.2 Frontend (React)

- **React 19 + Vite** for the pages, **React Router** for the URLs, **Tailwind CSS** for the look (light and dark mode).
- **Redux Toolkit** keeps what many pages need: who is logged in, the cart, the wishlist hearts.
- **One door for the server:** every call goes through `src/api/client.js` (Axios). Errors from the server come back as one standard shape (RFC 7807 `ProblemDetail`) and `describeError()` turns them into friendly sentences.
- **Guest cart:** a visitor's cart lives in the browser (localStorage). At login it is merged into the saved cart on the server.

2.3 Backend (Spring Boot) - a modular monolith

> **Real-life example:** A mall has one building, one entrance and one security guard, but every shop has its own locked storeroom. A shop may ask another shop's counter for something, but may never walk into its storeroom.

TriVoKo is **one Spring Boot application** (one building) split into **modules** (shops): `catalog`, `seller`, `cart`, `order`, `payment`, `coupon`, `flashsale`, `returns`, `review`, `wishlist`, `search`, `notify`, `analytics`, `admin`, `user`, `auth`, `health`.

The rule: a module may call another module's **service**, never its **repository** or its tables. An **ArchUnit** test checks this on every build - if someone breaks the rule, the build fails.

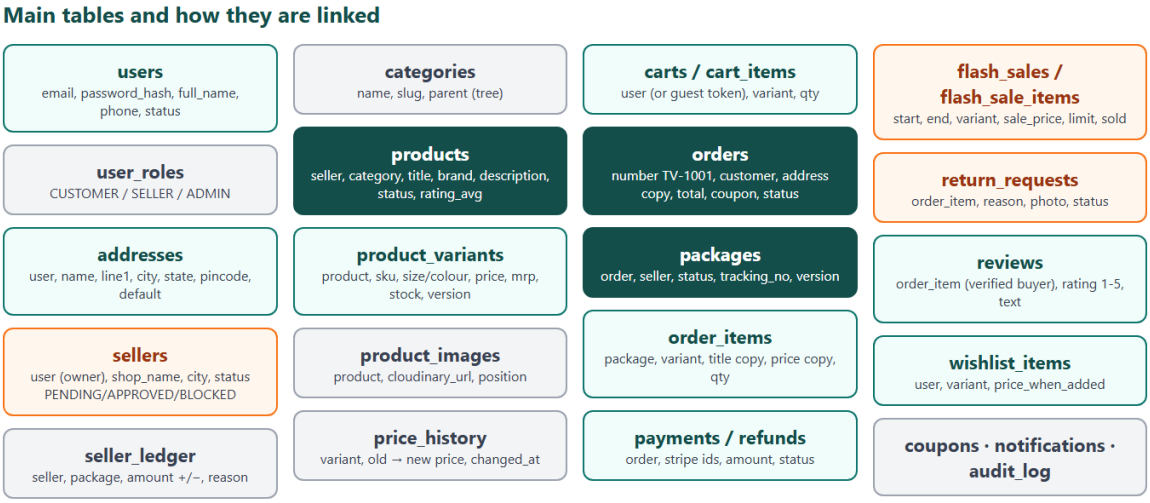
Why not microservices? One person, one small database, one deployment. Microservices would add network calls, many deployments and distributed transactions with no benefit at this size. The module rule keeps the code ready to be split later if it ever must be.

Inside each module the layers are always the same:

```
Controller  (URL, JSON in/out, validation)
    |
Service     (the business rules, @Transactional)
    |
Repository  (Spring Data JPA)  ->  MySQL
```

2.4 Database

- **MySQL 8.4.** Money is `DECIMAL`, never `double`.
- **Flyway owns the schema:** files `V1__...sql` to `V12__...sql` build every table. Hibernate is set to `validate` - it only checks that the Java classes match. An applied migration is never edited; a change is a new file.
- **Important tables:** `products` and `product_variants` (one row per size/colour with its own price and stock), `orders` -> `packages` (one per seller) -> `order_items`, `payments`, `seller_ledger`, `flash_sale_items` + `flash_sale_claims`, `return_requests`, `price_history`, `wishlist_items`, `reviews`, `audit_log`.



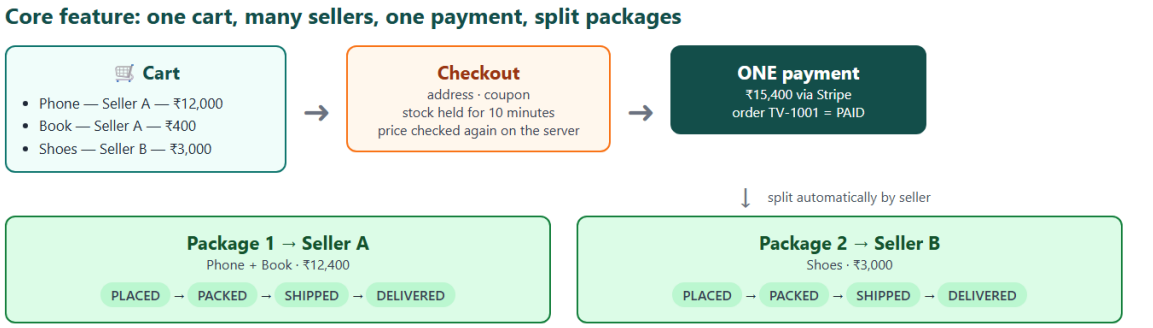
Database tables

2.5 Security

- **Login:** password stored with **BCrypt**. On login the server puts a **JWT** in an **HttpOnly, Secure, SameSite=Lax** cookie for 8 hours. JavaScript cannot read it, so an injected script cannot steal it.
- **Roles are read from the database on every request** (not trusted from the token), so when the admin blocks someone, it works on their very next click.
- **CSRF:** every POST/PUT/DELETE must send the `XSRF-TOKEN` cookie value back in the `X-XSRF-TOKEN` header.
- **Login lock:** 5 wrong passwords = 15 minutes locked, counted per email and per visitor address.
- **Ownership checks:** a seller can only touch their own products and packages (`@sellerAccess`), otherwise 403.
- **Headers on the live site:** Content-Security-Policy (scripts only from our own address), HSTS, nosniff, no framing.
- **Secrets** (database password, JWT secret, API keys) are environment variables - never in Git. `gitleaks` checks every commit.

3. The important flows, step by step

3.1 Buying from two sellers with one payment



The customer sees two tracking timelines inside one order. Seller A never sees Seller B's items or the customer's other purchases. Each package keeps its own money line: 90% to the seller's earnings, 10% TriVoKo fee.

The order split

1. The cart is priced **on the server** every time (`CartPricing`) - the browser's prices are never trusted.
2. **Checkout preview** shows one box per seller (= one package), the coupon and delivery: free from Rs 499 per package, else Rs 40.
3. **Pay** (`POST /api/orders`): the server checks the prices again (409 if they changed), **holds the stock** with one atomic `UPDATE ... WHERE stock >= qty` per item, and creates one order with one package per seller.
4. The **coupon is shared across the items by price**; the last item takes the rounding remainder, so the shares always add up exactly. Each item stores its share - this makes later refunds exact.
5. The customer pays on **Stripe** (test mode) or the built-in test payment page. Stripe tells the server by a **webhook** (signature checked; a duplicate message changes nothing).
6. Paid: order PAID, packages PLACED, bell + email + PDF invoice. Not paid in 10 minutes: EXPIRED, stock goes back.

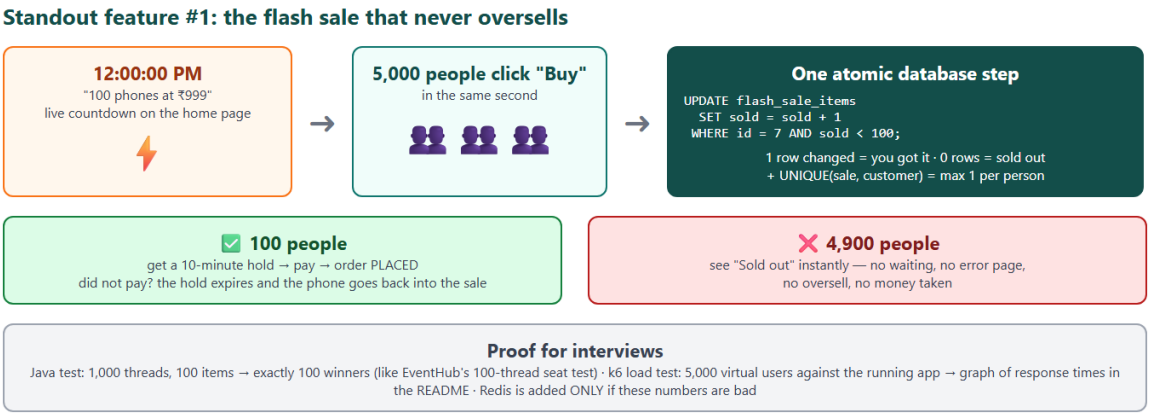
> **Real-life example:** One bill at the supermarket, but the bags are packed at different counters - each counter (seller) packs and sends its own bag.

3.2 A seller ships a package

```
PLACED --pack--> PACKED --tracking no.--> SHIPPED --> DELIVERED
   |                                                    (+90% to the seller's ledger)
   +--- before shipping ---> CANCELLED (package refunded, stock back)
```

Every move goes through one method that knows the allowed next states; an illegal move is 409. If two people change the same package at once, `@Version` (optimistic locking) makes one of them get 409 instead of both "winning".

3.3 The flash sale



Flash sale

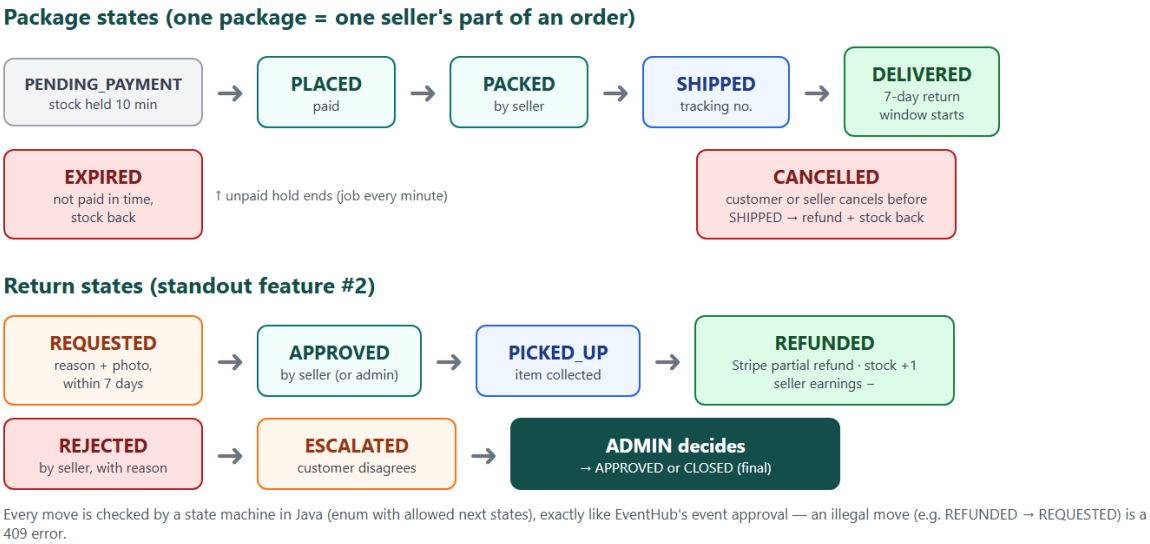
The whole claim is **one SQL statement**:

```
UPDATE flash_sale_items SET sold = sold + 1
 WHERE id = ? AND sold < quantity AND <the sale is live>
```

MySQL locks the row, checks and adds in one step. 1 row changed = you won; 0 = sold out. Three more guards: `UNIQUE (item_id, user_id)` = one per person; `CHECK (sold <= quantity)` = the database itself refuses an oversell; a job gives back units not paid within 10 minutes. A rate limit (5 claims per user, 200 per address, every 10 s) answers 429.

Proof: k6 with 5,000 users pressing at the same instant: exactly 100 winners, 4,900 "sold out", 0 errors, 0 oversell.

3.4 A return and its partial refund

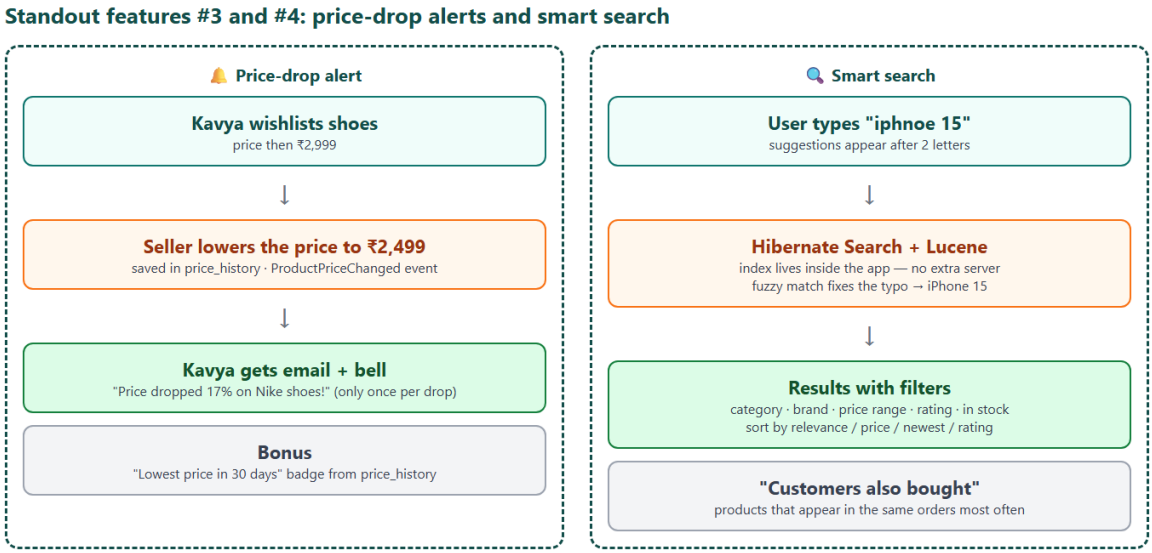


Return states

```
REQUESTED -> APPROVED -> PICKED_UP -> REFUNDED
    |            (seller silent 2 days = approved automatically)
    +-> REJECTED -> ESCALATED (once) -> admin decides
```

Refund = item total - that item's coupon share (delivery is not refunded). Stripe is called with the idempotency key `refund-return-<id>`, so ten clicks still refund once. In the same transaction: stock +1, the seller's ledger gets **-90%** of the refund, the customer gets a bell + email.

3.5 Price-drop alerts



Price alerts and search

A price change writes a `price_history` row and an event. After the save is committed, the event only **marks** the wishlist rows of that product. Every 10 minutes a job looks at marked rows and sends **one** alert if today's price is at least 1% below the last price the customer heard from us. Five price edits in a minute = one alert.

3.6 Smart search

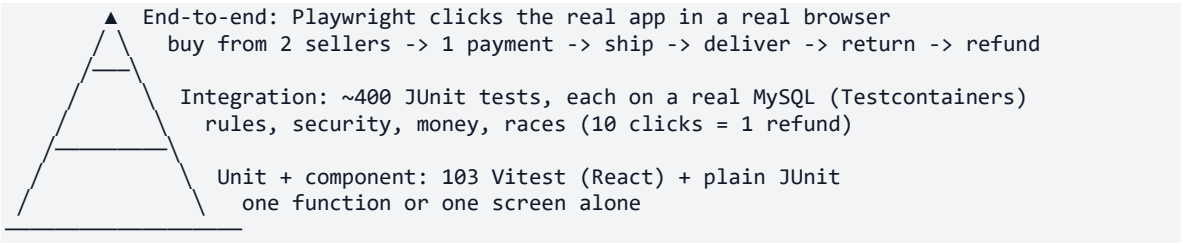
Lucene (through Hibernate Search) finds and ranks up to 500 product ids, allowing typos by word length (1 typo from 5 letters, 2 from 8, first letter must be right). MySQL then applies the live filters - in stock, category, brand, price - and counts the facets. Lucene for the text, MySQL for the facts that change every minute.

3.7 One-click demo logins (launch)

On the live site the login page offers "Try it as Ravi" (customer) and "Try it as Chennai Mobiles" (seller). It is the same HttpOnly cookie as a normal login. It is never offered for the admin, it is switched off unless `DEMO_ONE_CLICK=true`, and a blocked demo account stays blocked.

4. How we know it works

4.1 The test pyramid



> **Real-life example:** A car factory tests every bolt (unit), then the engine on a bench (integration), then drives the whole car on the road (end-to-end). Many cheap bolt tests, a few expensive road tests.

Why a real MySQL in tests? An in-memory database (H2) behaves differently: locking, `CHECK` constraints and some SQL. The flash-sale and stock tests depend on real MySQL locking, so **Testcontainers** starts a throwaway MySQL 8.4 in Docker for the test run.

Race tests: `StockConcurrencyTest` (100 customers, last 10 units -> exactly 10 orders), the flash-sale claim test, `ReturnRefundTest` (10 clicks at once -> 1 refund).

4.2 CI - GitHub Actions on every push

Job	What it checks
Backend	`mvnw verify`: all JUnit tests, ArchUnit module rule, JaCoCo coverage (fails below 80% of the service code)
Frontend	lint (oxlint), Vitest, production build
E2E	the Playwright demo story + accessibility (axe, WCAG 2.1 AA) in light/laptop and dark/phone
Security scan	`npm audit` (high), gitleaks (no secret in any commit)
Uptime (launch)	every 10 minutes: is the live API UP? a failure e-mails the owner

4.3 Performance and accessibility

- **k6 load test** (Phase 7): 5,000 users at the same instant -> exactly 100 winners, 0 errors. Speed in that burst (p95 about 1 s on one laptop) is above the 500 ms target - correctness first; decision D8 explains when Redis would be added.
- **Lighthouse** on a phone profile: Performance 92, Accessibility 100, Best practices 96, SEO 100 (self-hosted font, lazy-loaded pages, no layout jump).

5. How it is launched

5.1 The three free services

Piece	Service	Notes
Database	Aiven free MySQL (Bengaluru)	Flyway creates all tables on the first start
API	Render free, Docker (Singapore)	`render.yaml` Blueprint; sleeps after 15 idle minutes, so the uptime check pings it
Shop	Vercel	`vercel.json`: `/api/*` to Render, other paths to `index.html`, security headers

5.2 Making Spring Boot fit in 512 MB

- **Multi-stage Docker image:** stage 1 builds the JAR with Maven, stage 2 holds only the Java runtime and the JAR - smaller, and no source code on the server. It runs as a normal user, not root.
- **Java options:** `-XX:MaxRAMPercentage=75 -XX:+UseSerialGC -Xss512k` - Java may use 75% of the memory, the simplest garbage collector, smaller thread stacks.
- **prod profile:** 40 web threads (not 200), 5 database connections, search index in `/tmp`.
- Measured in Docker before going live: about 400-430 MB after a full purchase.

5.3 Production settings

- Everything comes from **environment variables**: `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `JWT_SECRET`, `FRONTEND_URL`, `CORS_ALLOWED_ORIGINS`, `DEMO_PASSWORD`, Cloudinary and Stripe keys. The JWT secret has **no fallback** in prod - the server refuses to start without it.
- `server.forward-headers-strategy: framework`: behind Vercel and Render the app sees the visitor's real address (from `X-Forwarded-For`) and knows the request was HTTPS - needed for the login lock and the Secure cookie.

5.4 What a launch taught us

- A Render Blueprint form can drop secret fields that were typed in it - always **check the list of variables** after creating the service.
- "Access denied for user" from MySQL means the address was right and the password was wrong (or missing) - read the log line before changing anything.

6. What I would do next

- **Redis** for the flash-sale counter and the rate limit, if the burst must be faster than 500 ms or the app runs as several copies.
- **Elasticsearch / OpenSearch** instead of the in-app Lucene index if the app runs as several copies.
- **Real e-mail** through Brevo (the code is ready; only the SMTP settings are missing on the live server).
- **Seller payouts** (the ledger already knows each seller's balance).
- **Images resized on upload** and a CDN for the product photos.

7. Viva and interview questions

Read the question, answer it **aloud in your own words**, then compare with the short answer. You do not need the exact words - you need the idea and one example from TriVoKo.

7.1 The project

Q1. Tell me about your project.

Use the two-minute answer in chapter 1.2: problem, product, four standouts, quality, live.

Q2. Why did you build a marketplace and not a simple shop?

A marketplace has harder, more interesting problems: one payment split across many sellers, each seller's earnings, and stock that many people buy at once. Those are the problems real companies have.

Q3. What was the hardest part?

Keeping money exact. The coupon is shared across the items by price, each item stores its share, and a return refunds exactly that item minus its share. Money, stock and the seller ledger must always agree, and tests check each case.

Q4. Which part did you write yourself?

The three key pieces of logic: the order split, the flash-sale claim and the return state machine. (Answer honestly about how you built and reviewed them.)

7.2 Java and Spring Boot

Q5. What is a modular monolith?

One application, one deployment, but split into modules with clear borders. In TriVoKo a module may call another module's service, never its repository or tables. An ArchUnit test fails the build if the rule is broken.

Q6. Why not microservices?

At this size they add cost without benefit: network calls, many deployments, distributed transactions. The module borders keep the option open to split later.

Q7. What does @Transactional do?

It runs the method in one database transaction: all changes are saved together or none are. Example: a refund puts stock back, writes the minus line in the seller ledger and changes the return status in one transaction.

Q8. What is the N+1 problem and how did you avoid it?

Loading 24 products and then making one extra query for each product's variants = 25 queries. TriVoKo sets `hibernate.default_batch_fetch_size: 50`, so the variants of 50 products come in one query. Ratings are stored on the product row, so cards need no extra query.

Q9. Why Flyway and not ddl-auto=update?

Flyway gives a numbered, reviewed history of every schema change, the same on every machine. `ddl-auto=update` guesses changes, can never drop or rename safely, and leaves no history. TriVoKo uses Flyway to build and Hibernate only to `validate`.

Q10. How do you handle errors in the API?

Services throw a few own exceptions (`ResourceNotFoundException`, `BusinessRuleException` ...). One `GlobalExceptionHandler` turns them into RFC 7807 `ProblemDetail` JSON with the right status (404, 409, 400). The React app reads `detail` and shows a friendly message.

Q11. What is optimistic locking? Where did you use it?

A `@Version` column: an update succeeds only if the version is still the one you read. If two people change the same package at once, one wins and the other gets 409 - no silent overwrite. Used on packages and returns.

7.3 Database and concurrency

Q12. How does the flash sale never oversell?

One atomic statement: `UPDATE flash_sale_items SET sold = sold + 1 WHERE id = ? AND sold < quantity AND <live>`. MySQL locks the row and checks and adds in one step. 1 row changed = won, 0 = sold out. Plus `UNIQUE(item, user)` and `CHECK (sold <= quantity)`.

Q13. Why not read the stock in Java, check it, then save?

Two requests can both read "1 left" before either saves, and both buy. That is a race condition. The check must happen inside the same statement that changes the number.

Q14. How did you prove it?

A Java test with many threads, and a k6 load test: 5,000 users at the same instant, 100 units, exactly 100 winners, 0 errors, and the database agreed afterwards.

Q15. Why is money DECIMAL and not double?

`double` cannot store values like 0.1 exactly, so sums drift by tiny amounts. `DECIMAL` (and `BigDecimal` in Java) is exact - required for money.

Q16. What indexes did you add and why?

Examples: `price_history (variant_id, changed_at)` for "lowest price in 30 days", a unique key on reviews (one per customer per product), and an index for the admin activity log. An index is like the index at the back of a book - find the page without reading every page.

7.4 Security

Q17. Where is the login token stored and why?

In an HttpOnly, Secure, SameSite=Lax cookie. JavaScript cannot read it, so a script injected into the page cannot steal it. That is safer than localStorage.

Q18. What is CSRF and how is it stopped?

Another website making the user's browser send a request to our site with the user's cookie. TriVoKo requires a token from the `XSRF-TOKEN` cookie in the `X-XSRF-TOKEN` header on every POST/PUT/DELETE - another site cannot read our cookie, so it cannot send the header.

Q19. How do passwords stay safe?

BCrypt hashes (slow on purpose, with a salt). Wrong email and wrong password give the same message, and 5 wrong tries lock that login for 15 minutes.

Q20. Why read the roles from the database on every request?

So a block or a role change works at once. If the roles were trusted from the token, a blocked user could keep using the site until the token expired (8 hours).

Q21. How do you stop a seller from editing another seller's product?

Every seller URL checks ownership (`@sellerAccess`): another shop's product or package = 403. Tests check this for each kind of object.

Q22. Where are the secrets?

Only in environment variables on the server and in a `.env` file on the laptop that is never committed. gitleaks scans every commit in CI.

7.5 Payments

Q23. How does Stripe tell you the payment worked?

A webhook: Stripe calls `/api/payments/stripe/webhook`. The server checks the signature, then marks the order paid. If the same message arrives twice, the second changes nothing (idempotent). The return page also asks Stripe directly, in case the webhook is late.

Q24. How do you make sure a refund never happens twice?

Each refund uses an idempotency key (`refund-return-<id>`) - Stripe returns the first result for a repeated key. And `@Version` stops a second click before any money moves.

Q25. What happens if the customer never pays?

The stock is held for 10 minutes. A job marks the order EXPIRED and gives the stock back. A late payment re-takes the stock or is refunded.

7.6 React

Q26. Why Redux Toolkit?

For state many pages share: the logged-in user, the cart badge, the wishlist hearts. Page-only state stays in the component with `useState`.

Q27. How does the guest cart work?

It is stored in localStorage. After login the lines are sent to `/api/cart/merge` and joined with the saved cart; prices always come from the server.

Q28. How did you make the site accessible?

Labels on every field, keyboard support (the search box is an ARIA combobox), colour contrast fixed until axe reported no serious problems, in light and dark mode.

7.7 Testing and DevOps

Q29. What is Testcontainers?

A library that starts real services (here MySQL 8.4) in Docker for the tests, so tests run against the same database as production.

Q30. What is in your CI pipeline?

Backend build and tests with coverage gate, frontend lint/tests/build, Playwright end-to-end with accessibility, and a security scan - on every push. Merge only when all are green.

Q31. What does the Dockerfile do?

Two stages: build the JAR with Maven, then copy only the JAR into a small Java runtime image that runs as a non-root user. Settings come from environment variables, so the same image runs anywhere.

Q32. How did you deploy it for free?

React on Vercel, Spring Boot in Docker on Render, MySQL on Aiven. Vercel passes `/api` calls to Render so everything is one address. An uptime check keeps the free server awake.

Q33. Your app sits behind proxies. What problem does that create?

The app would see the proxy's address for every visitor, so one person's wrong passwords would lock everyone. `forward-headers-strategy: framework` reads the real address from `X-Forwarded-For`.

7.8 Smart search and alerts

Q34. How does the search handle typos?

Lucene fuzzy matching: 1 typo allowed from 5 letters, 2 from 8, and the first letter must be right - so "iphnoe" finds iPhone but does not drag in "phone" for every word.

Q35. Why both Lucene and MySQL in one search?

Lucene is best at text and ranking; MySQL has the facts that change every minute (stock, price, blocked shops). Lucene returns ranked ids, MySQL filters and counts them.

Q36. How do price alerts avoid spam?

The price-change event only marks wishlist rows. A job every 10 minutes sends one alert if the latest price is at least 1% below the last price the customer was told. Five edits in a minute = one alert.

7.9 Reflection

Q37. What would you change if you started again?

Answer honestly. Ideas: decide the hosting earlier, add Redis only if measurements demand it (as done), write the end-to-end test from Phase 3 instead of Phase 11.

Q38. What did you learn?

Pick two: thinking about races and money exactly; testing against a real database; production settings (memory, proxies, secrets); writing decisions down (`docs/decisions.md`).

Q39. How would it scale to 100 times more users?

Several copies of the app behind a load balancer; then move what lives inside one copy - the search index and the rate limits - to shared services (Elasticsearch, Redis); a read replica for MySQL; photos on a CDN.

Q40. Which part are you most proud of, and why?

Your own answer - see chapter 1.4.